

.NET

.NET AI Agent Engineering Roadmap

From Junior to Senior – Build, Learn, Ship Real AI Agents

One Ecosystem | Real Projects | Hands-on Code | From Local to Cloud | Production Ready



Build the
Next Generation
of AI with .NET

Learn
Build
Grow

10+1 Steps to Become a .NET AI Agent Expert

Master the complete stack – from models to multi-agent systems

Same Skills
Different Levels
Endless Opportunities!

1 .NET Foundation & Basics

Core skills for every level

- C#, OOP, async/await
- ASP.NET Core (Minimal APIs, DI)
- EF Core, SQL Server, Redis
- Testing, Git, Docker, CI/CD

```
// Minimal API
var app = WebApplication.Create();
app.MapGet("/", () => "Hello .NET!");
app.Run();
```

Junior

2 Development Environment & Model Setup

Build locally, scale to cloud

- Visual Studio / VS Code
- Ollama / Foundry Local (local models)
- Azure AI Foundry (cloud models)
- Docker, Dev Containers

```
// Using a local model via Ollama
IChatClient client = new OllamaChatClient(
    new Uri("http://localhost:11434"),
    "llama3.2");
```

Junior → Mid

3 Microsoft.Extensions.AI & Model Integration

Unified abstraction for any model

- IChatClient & Embeddings
- Streaming responses
- Structured output (JSON)
- Switch between providers

```
IChatClient client =
    new OpenAIChatClient(
        model: "gpt-4o",
        apiKey: config["OpenAI:Key"]);
var response = await client
    .GetResponseAsync("Explain RAG.");
```

Mid

4 Microsoft Agent Framework

Build powerful AI agents

- Create agents, sessions, middleware
- Graph-based workflows
- State management
- Human-in-the-loop

```
var agent = chatClient.CreateAgent(
    instructions: "You are an HR assistant.");
var result = await agent.RunAsync(
    "What is our leave policy?");
Console.WriteLine(result);
```

Mid

5 Context & Memory

Make agents remember

- Conversation history
- Short-term & long-term memory
- Semantic memory (vector search)
- Session state & user preferences

```
var memory = new MemoryBuilder()
    .WithVectorStore(vectorStore)
    .WithChatHistory()
    .Build();
```

Mid

6 Tool Calling & Function Calling

Give agents real-world skills

- Call APIs, databases, file systems
- Validate, authorize and secure
- Handle errors, retries, approvals
- Use MCP for external tools

```
[Description("Get employee details")]
public async Task<Employee>
    GetEmployee(string id)
{
    return await _db.Employees
        .FirstOrDefault(e => e.Id == id);
}
```

Mid → Senior

7 Semantic Kernel

Enterprise AI orchestration

- Kernel, plugins, prompt templates
- Function calling
- Memory & embeddings
- Existing enterprise apps & migration to Agent Framework

```
var kernel = Kernel.CreateBuilder()
    .AddOpenAIChatCompletion("gpt-4o", key)
    .Build();
kernel.Plugins.AddFromType(HRPlugin);
```

Mid → Senior

8 Semantic Search & Vector Database

Power your RAG with your data

- Embeddings generation
- Vector stores (Azure AI Search, Qdrant, Pinecone, pgvector, Chroma, etc.)
- Hybrid search (keyword + vector)
- Reranking, filters, metadata

```
var embedding = await embeddingClient
    .GenerateEmbeddingAsync("policy document");
await vectorStore.UpsertAsync(
    new VectorRecord("doc1", embedding,
        metadata: new { source = "hr.pdf" }));
```

Senior

9 MCP Server & Client Architecture

Standardized tool & resource access

- Build MCP servers in .NET
- MCP client integration
- Expose tools, resources, prompts
- Secure, scalable, interoperable

```
var server = McpServer.Create()
    .AddTool("getWeather", GetWeather)
    .Build();
await server.RunAsync();
```

Senior

10 RAG & Agentic RAG

Ground agents with your data

- Document ingestion & chunking
- Semantic search (vector database)
- Hybrid retrieval & reranking
- Citations, source tracking
- Agentic RAG (agent decides when/how to search)

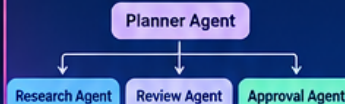
```
var results = await vectorStore.SearchAsync(
    "remote work policy", topK: 5);
var context = string.Join("\n",
    results.Select(r => r.Content));
var answer = await chatClient.GetResponseAsync(
    $"Use the context to answer: {context}");
```

Senior

11 Multi-Agent Orchestration & Workflows

Solve complex problems with teams

- Sequential, parallel, and routing workflows
- Multi-agent collaboration
- Task planning and dynamic delegation
- Human approval steps
- Durable execution & checkpoints

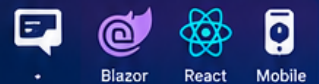


Senior

12 Agent Communication, AG-UI & User Experience

Connect agents and users seamlessly

- A2A (Agent-to-Agent) communication
- Event-driven messaging (gRPC, HTTP, Service Bus)
- AG-UI for interactive web/mobile UI
- Real-time streaming, tool progress
- Human-in-the-loop experiences



Senior

13 Debugging, Observability & DevUI

Understand, trace and improve

- Microsoft Agent Framework DevUI
- Distributed tracing with OpenTelemetry
- Logs, metrics, token usage, cost tracking
- .NET Aspire for local orchestration
- Evaluate and test agents

```
builder.AddProject<Projects.AI>("agent")
    .WithOpenTelemetry()
    .WithOtlpExporter();
```

Senior

14 Deployment & Production

From prototype to production

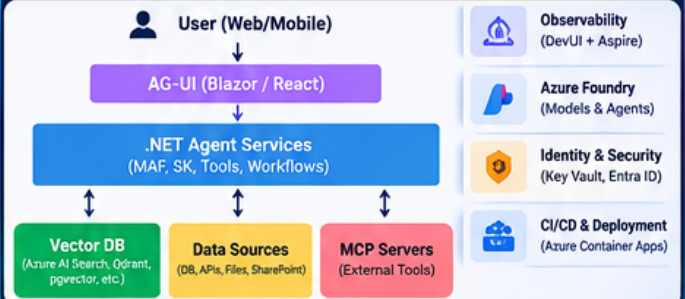
- Azure Container Apps / Kubernetes
- Microsoft Foundry (models, agents, evals)
- Managed Identity, Key Vault, security
- Scaling, caching, rate limiting
- AI evaluation & regression testing
- Cost and latency monitoring

```
app.MapPost("/chat", async (ChatRequest req) =>
{
    var result = await agent.RunAsync(req.Message);
    return Results.Ok(result);
});
```

Senior

Complete Architecture

All pieces working together



Example End-to-End Project

- Enterprise AI Assistant (HR / IT / Knowledge)
- Chat with documents (RAG)
- Uses company tools (MCP)
- Multi-agent workflow with approvals
- Web UI with AG-UI
- Deployed on Azure with monitoring

Recommended Tech Stack



Same Developer
Bigger Possibilities

Learn Today. Build Tomorrow.

.NET AI Agent Engineering Roadmap

From Junior to Senior Developer

A structured, project-based guide to building production-ready AI agents with C# and the Microsoft .NET ecosystem.

FOUNDATION	AI APPLICATIONS	AGENT ENGINEERING
C# • ASP.NET Core Async/Await • APIs SQL • Testing • Docker	LLMs • MEAI Local AI • Foundry Embeddings • Prompts	MAF • Tools • Memory Semantic Search • Vector DB RAG • MCP
ADVANCED SYSTEMS	USER EXPERIENCE	PRODUCTION
Workflows • Multi-Agent A2A • Human Approval	AG-UI • Blazor • React Streaming • Events	DevUI • Aspire OpenTelemetry • Azure Security • Evaluation

Learning order: .NET foundations → LLM integration → agents → context and memory → tool calling → Semantic Kernel → embeddings and semantic search → vector databases → RAG → MCP → multi-agent workflows → AG-UI → observability → deployment.

Roadmap Contents

- 1. .NET Foundation
- 2. AI and Model Integration
- 3. Local AI, Microsoft Foundry, and Development Environment
- 4. Microsoft Agent Framework
- 5. Context, Memory, and State
- 6. Tool Calling and Function Calling
- 7. Semantic Kernel
- 8. Embeddings, Semantic Search, and Vector Databases
- 9. RAG and Agentic RAG
- 10. MCP Server and Client Architecture
- 11. Agentic Workflows and Multi-Agent Orchestration
- 12. Agent Communication
- 13. AG-UI Integration
- 14. DevUI, Debugging, and Observability
- 15. Production Deployment
- 16. Technology Stack and Capstone Project

□ The Complete .NET AI Agent Engineering Roadmap — From Junior to Senior

How do you become an AI Agent Engineer using C# and the Microsoft .NET ecosystem?

You do not need to learn every AI framework at once. Start with strong software engineering, understand how LLMs work, and gradually build agents that can use tools, search knowledge, remember context, and collaborate.

Here is a structured roadmap with the technologies, concepts, and projects to learn at each stage.

□ LEVEL 1 — .NET FOUNDATION

1. Master C# and ASP.NET Core

Learn:

- C# OOP, interfaces, delegates, generics, LINQ
- Async/await, Task, concurrency, cancellation tokens
- ASP.NET Core Web API, Minimal API, dependency injection
- Entity Framework Core, SQL Server, Redis
- Authentication, authorization, testing, Git, Docker

□ Project: Build an Employee Management API with SQL Server, authentication, logging, and unit tests.

Goal: Become confident building production-quality .NET backend applications.

□ LEVEL 2 — AI & MODEL INTEGRATION

2. Understand LLM Fundamentals

Learn:

- Tokens and context windows
- Prompt engineering and system instructions
- Temperature and model selection
- Streaming and structured JSON output
- Embeddings and hallucinations
- AI model limitations, latency, and cost

3. Microsoft.Extensions.AI

Code example: Learn IChatClient, embedding abstractions, provider integration, and model-independent application design.

Code example: using Microsoft.Extensions.AI; IChatClient chatClient = /* configured AI provider */; var response = await chatClient.GetResponseAsync("Explain dependency injection in C#."); Console.WriteLine(response.Text);

□ Project: Build a chatbot and text summarization API.

□ LEVEL 3 — LOCAL AI, AZURE & DEVELOPMENT ENVIRONMENT

4. Develop and run AI locally

Tools to learn:

- Visual Studio / VS Code
- .NET SDK and NuGet

- Ollama
- Foundry Local
- Docker
- Microsoft Foundry

Understand how to use local models during development and Azure-hosted models in production.

□ Project: Create a model-switchable chatbot that can run locally and connect to Microsoft Foundry.

□ LEVEL 4 — MICROSOFT AGENT FRAMEWORK

5. Build your first AI Agent

An AI agent is more than a chatbot. It can plan or decide the next action, use tools, maintain state, and complete multi-step tasks.

Learn Microsoft Agent Framework (MAF):

- Agent creation and instructions
- Agent sessions and state
- Streaming responses
- Tool registration
- Middleware
- Graph-based workflows
- Human-in-the-loop

Code example: using Microsoft.Agents.AI; using Microsoft.Extensions.AI; IChatClient chatClient = /* configured provider */; AIAgent agent = chatClient.CreateAIAgent(instructions: """ You are an employee-support assistant. Use approved tools when necessary. """); var result = await agent.RunAsync("Explain our leave policy."); Console.WriteLine(result);

□ Project: Build an HR assistant that answers employee questions and retrieves approved company information.

□ LEVEL 5 — CONTEXT, MEMORY & STATE

6. Make your agents remember useful information

Learn the difference between:

- Conversation history
- Short-term memory
- Long-term memory
- Agent session state
- Semantic memory
- Persistent storage
- User preferences and privacy

Understand what information should be remembered, when it should expire, and how it should be retrieved.

□ Project: Build a productivity agent that remembers project preferences and retrieves relevant previous notes.

Important: Memory should be selective, secure, and controlled. Never store sensitive business information without proper authorization.

□ LEVEL 6 — TOOL CALLING & FUNCTION CALLING

7. Give agents real-world capabilities

Tools allow an agent to interact with business systems:

- SQL Server and databases
- REST APIs
- Payroll services
- File systems
- Ticketing systems
- External APIs

Learn:

- Tool schemas and typed parameters
- Argument validation
- Authorization
- Retries and timeouts
- Idempotency
- Human approval
- Error handling

Code example: using System.ComponentModel; public sealed class EmployeeTools { [Description("Gets an employee's leave balance.")] public int GetLeaveBalance(string employeeId) { // Validate and authorize before querying. return 12; } }

□ Project: Build a payroll assistant that can read salary information but requires manager approval before making changes.

□ LEVEL 7 — SEMANTIC KERNEL

8. Understand Semantic Kernel

Semantic Kernel is important for understanding existing enterprise AI applications and its concepts remain valuable when working with the wider Microsoft agent ecosystem.

Learn:

- Kernel and dependency injection
- Plugins and functions
- Prompt templates
- Function calling
- Agent abstractions
- Filters and middleware
- Embeddings and memory

□ Project: Build a plugin-based business assistant and understand how to migrate or integrate existing SK applications with newer agent architectures.

□ LEVEL 8 — EMBEDDINGS, SEMANTIC SEARCH & VECTOR DATABASES

9. Learn how AI searches knowledge by meaning

This is one of the most important topics for becoming an AI engineer.

Traditional keyword search looks for matching words.

Semantic search looks for related meaning using embeddings.

Example:

User asks:

"How many paid holidays do employees receive?"

A semantic search system can find a document titled:

"Annual Leave and Holiday Entitlement Policy"

Even if the exact words do not match.

Learn:

- Embeddings and vector representations
- Similarity search
- Cosine similarity and distance metrics
- Chunking and metadata
- Vector indexing
- Top-K retrieval
- Metadata filtering
- Hybrid search
- Reranking

10. Vector Databases

Learn how to store and search embeddings.

Technologies to explore:

- Azure AI Search
- PostgreSQL + pgvector
- Qdrant
- Redis Vector Search
- Pinecone
- Other vector-capable databases

For .NET, explore:

Microsoft.Extensions.VectorData

```
Code example: public sealed class KnowledgeRecord { public string Id { get; set; } = ""; public string Content { get; set; } = ""; public ReadOnlyMemory<float> Embedding { get; set; } }
```

The record represents content and its vector embedding. The actual storage and similarity-search implementation depends on your selected vector database.

□ Project: Build a company policy search engine that converts documents into embeddings and retrieves the most relevant passages.

□ LEVEL 9 — RAG & AGENTIC RAG

11. Retrieval-Augmented Generation (RAG)

RAG combines your own knowledge with an LLM.

Typical pipeline:

Documents → Chunking → Embeddings → Vector Database → Semantic Search → Context → LLM → Grounded Answer

Learn:

- Document ingestion
- Chunking strategies
- Embeddings
- Semantic search
- Vector database integration
- Hybrid retrieval
- Reranking
- Citations
- Access control
- Retrieval evaluation

Useful .NET libraries:

Microsoft.Extensions.DataIngestion

Microsoft.Extensions.VectorData

12. Agentic RAG

Traditional RAG retrieves information using a predefined pipeline.

Agentic RAG allows the agent to decide:

- Should I search?
- Which knowledge source should I use?
- What query should I generate?
- Do I need another search?
- Is the retrieved evidence sufficient?

□ Project: Build an enterprise policy assistant that searches documents, cites evidence, and uses agent-driven retrieval when the first search is insufficient.

□ LEVEL 10 — MCP SERVER & CLIENT ARCHITECTURE

13. Model Context Protocol (MCP)

MCP standardizes how AI applications connect to external tools and resources.

Learn:

- MCP server and client
- Tools, resources, and prompts
- Capability discovery
- Local and remote connections
- Authentication and authorization
- Secure tool execution

Architecture:

AI Agent → MCP Client → MCP Server → Database / API / Files

□ Project: Build a .NET MCP server exposing read-only employee data and connect it to an MAF agent.

□ LEVEL 11 — AGENTIC WORKFLOWS & MULTI-AGENT ORCHESTRATION

14. Build complex AI systems

Learn:

- Sequential workflows
- Concurrent workflows
- Conditional routing
- Handoff patterns
- Group chat
- Human-in-the-loop
- Checkpointing
- State management
- Error recovery

Example:

Expense Request → Extraction Agent → Policy Agent → Fraud Review → Human Approval

Do not use multiple agents just because it is fashionable. Use them when specialization, independent execution, or workflow separation provides a real benefit.

□ Project: Build an expense approval system with multiple specialized agents and a human approval step.

□ LEVEL 12 — AGENT COMMUNICATION

15. Make agents and services communicate

Learn:

- In-process typed messages
- HTTP and REST APIs
- gRPC
- Event-driven architecture
- MCP for tool interoperability
- A2A for agent-to-agent communication
- Message contracts and correlation IDs

□ Project: Deploy a research agent and summarization agent as separate ASP.NET Core services.

□ LEVEL 13 — AG-UI INTEGRATION

16. Connect agents to interactive user interfaces

Learn AG-UI and integrate agents with:

- Blazor
- React
- ASP.NET Core
- Streaming chat

- Tool progress
- Approval forms
- Agent state and events

□ Project: Build a Blazor or React interface where users can watch an agent execute tools and approve sensitive actions.

• LEVEL 14 — DEBUGGING, DEVUI & OBSERVABILITY

17. Understand what your agent is doing

Learn:

- Microsoft Agent Framework DevUI
- Agent conversations and tool execution
- .NET Aspire
- OpenTelemetry
- Structured logging
- Distributed tracing
- Token usage and cost
- Latency and failure analysis
- AI evaluation and regression testing

Use .NET Aspire to orchestrate your multi-service application and observe its components during development.

□ Project: Run your multi-agent application with Aspire and trace a request from the UI through the agent, MCP server, database, and model.

• LEVEL 15 — PRODUCTION AI ENGINEERING

18. Deploy reliable AI systems

Learn:

- Docker and CI/CD
- Microsoft Foundry
- Azure Container Apps
- Azure identity and managed identities
- Key Vault and secrets management
- Redis and persistent state
- Rate limiting and retries
- Prompt injection defenses
- Tool permissions and data protection
- Cost controls and caching
- Load testing and AI evaluations

□ Project: Deploy your complete enterprise assistant to Azure with monitoring, authentication, and automated tests.

□ THE COMPLETE .NET AI TECHNOLOGY STACK

.NET / C# → Core engineering

ASP.NET Core → APIs and backend

Microsoft.Extensions.AI → Model abstractions

Microsoft Agent Framework → Agents and workflows

Semantic Kernel → Plugins, functions, existing enterprise applications

Ollama / Foundry Local → Local AI

Microsoft Foundry → Cloud AI platform

Microsoft.Extensions.DataIngestion → Document preparation

Microsoft.Extensions.VectorData → Vector data abstractions

Azure AI Search / pgvector / Qdrant → Semantic and vector search

MCP → Tool interoperability

A2A → Agent-to-agent communication

AG-UI → Agent-to-user experiences

.NET Aspire → Distributed application orchestration

OpenTelemetry → Observability

Docker + Azure → Production deployment

□ FINAL CAPSTONE PROJECT

Build a complete Enterprise Employee AI Assistant:

□ C# + ASP.NET Core

□ Microsoft Agent Framework

□ Local AI + Microsoft Foundry

□ Context and persistent memory

□ Tool calling

□ Semantic search

□ Embeddings and vector database

□ Hybrid search + RAG

□ Agentic RAG

□ MCP server and client

□ Multi-agent workflows

□ Human approval

□ A2A communication

□ AG-UI with Blazor or React

□ DevUI + .NET Aspire + OpenTelemetry

□ Docker + Azure deployment

Career advice:

Junior developers should focus on C#, APIs, AI fundamentals, and simple tools.

Mid-level developers should learn agents, memory, semantic search, vector databases, RAG, MCP, and workflows.

Senior developers should focus on architecture, security, evaluation, observability, scalability, cost, and production reliability.

The goal is not to use the maximum number of frameworks.

The goal is to build AI systems that solve real business problems.

□ Learn the fundamentals. Build projects. Measure results. Ship reliable AI.

\#DotNet #CSharp #AIAgents #MicrosoftAgentFramework #SemanticKernel #MCP #RAG #SemanticSearch
#VectorDatabase #Azure #MicrosoftFoundry #Aspire #GenerativeAI #SoftwareEngineering #AIEngineering

Career Guidance

- Junior developers: focus on C#, APIs, AI fundamentals, simple tools, testing, and deployment basics.
- Mid-level developers: learn agents, memory, semantic search, vector databases, RAG, MCP, and workflows.
- Senior developers: focus on architecture, security, evaluation, observability, scalability, cost, and production reliability.
- The goal is not to use the maximum number of frameworks. The goal is to build AI systems that solve real business problems.